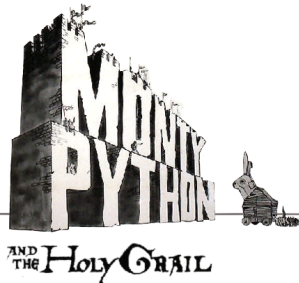


Python II



Python II

Reading a File

First we have to open the file for reading:

```
fileHandle = open('yellow_snow.txt')
```

Now we have a file handle „fileHandle“ pointing to the file.

Reading and printing the whole file is easy:

```
print fileHandle.read()
```

When we don't need the file anymore, we close it:

```
fileHandle.close()
```

Reading and Writing Files

Files are seen as data streams.

There are downstreams (in-going, reading) and upstreams (outgoing, writing).

Data streams:
Screen, keyboard, files, network connections

Reading and Writing to a File

We opened the file for reading with

```
fileHandle = open('yellow_snow.txt')
```

alternatively, we could have done it with:

```
fileHandle = open('yellow_snow.txt', 'r')
```

Opening a file for writing is analogue:

```
fileHandle = open('yellow_snow_copy.txt', 'w')
```

Python II

Reading and Writing

```
fobj = open("yellow_snow.txt")
for line in fobj:
    print line
fobj.close()
```

Reading and Writing:

```
fobj_in = open("yellow_snow.txt")
fobj_out = open("yellow_snow2.txt", "w")
i = 1
for line in fobj_in:
    print line.rstrip()
    fobj_out.write(str(i) + ": " + line)
    i = i + 1
fobj_in.close()
fobj_out.close()
```

Various ways of Reading a File

read entire file into string:

```
open('file').read( )
```

read next N bytes into string:

```
open('file').read(N)
```

read entire file into list of lines:

```
open('file').readlines( )
```

read next line, through '\n':

```
open('file').readline( )
```

Reading „Line by Line“ / char by char

```
fileHandle =
open("examples/yellow_snow.txt", "r")
for line in fileHandle:
    print line.rstrip()
fileHandle.close()
```

and now we read character by character:

```
fileHandle = open("yellow_snow.txt", "r")
char = fileHandle.read(1)
while char:
    print char,
    char = fileHandle.read(1)
fileHandle.close()
```

Using readline()

```
fh = open("yellow_snow.txt")
```

```
line = "no line"
while line:
    line = fh.readline()
    print (line.rstrip())
```

```
fh.close()
```

Empty lines contain at least a “\n”, so this script loops over the whole file.

Python II

File Scanner

```
def scanner(name, function):  
    file = open(name, 'r')  
    while True:  
        line = file.readline()  
        if not line: break  
        function(line)  
    file.close()
```

de'pickle

A file with a pickled object can be read back (deserialized)
`pickle.load(file)`



`pickle.load` recognizes automatically the protocol which had been used when saving the file.

Get yourself into a pickle

`pickle` is a module to serialize objects.

These objects can later (e.g. in another program) be automatically deserialized



```
pickle.dump(obj, file[,protocol])
```

A serialized version of the object `obj` will be written to the file

`file` (a handle)

`protocol` (0=ascii, 1=compact, not readable, 2=opt.

classes)

A Simple Example

```
import pickle  
data = (1.4, 42)  
output = open('data.pkl', 'w')  
pickle.dump(data, output)  
output.close()
```

The content of `data.pkl` will look like this:

```
(F1.3999999999999999  
I42  
tp0  
.
```

Python II

Reading the Pickled File

With `pickle.load` we can reread the pickled version.

```
>>> import pickle
>>> f = open("data.pkl")
>>> data = pickle.load(f)
>>> print data
(1.3999999999999999, 42)
>>>
```

Some Exceptions

```
>>> 1 / 0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: integer division or modulo by zero
>>> Monty * Python
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'Monty' is not defined
>>> "42" + 12
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: cannot concatenate 'str' and 'int' objects
>>> open("spam.txt", "r")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IOError: [Errno 2] No such file or directory:
'spam.txt'
```

Handling Exceptions

Errors occurring during execution of a script are called exceptions. They are not necessarily fatal as they can be handled in programs.

Java and C++ use

„try ... catch“ to handle exceptions and
„throw“ to generate them,
while Python uses

„try ... except“ to handle exceptions and
„raise“ to generate them

Nearly every module in Python uses exceptions.

If you don't catch an exception the entire program will crash.



Handling Exceptions

Let's have a look at how to handle exceptions.

The following example asks the user for input until a valid integer has been entered.

```
while True:
    try:
        x = int(raw_input("Please enter a number:
"))
        break
    except ValueError as err:
        print("error message: ",err)
        print("No valid number. Try again!")
```

The variable `err` will be automatically removed after finishing the try except statement.

Python II

Functional Principle of try-except

First, the try clause between the try and the except keywords is executed.

If no exception occurs, the except clause is skipped.

If an exception occurs during execution of the try clause, the rest of the clause is skipped. If the type of the exception matches the exception named after the except keyword, the except clause is executed, and then execution continues after the try statement.

If an exception occurs which does not match the exception named in the except clause, it is passed on to outer try statements; if no handler is found, it is an unhandled exception and execution stops with a message.

The optional else Clause

The try-except statement can have an optional else clause.

If it is present, it must follow all except clauses.

It is useful for code that must be executed, if the try clause does not raise an exception.

```
try:
    f = open(arg, 'r')
except IOError:
    print 'cannot open', arg
else:
    print arg, 'has', len(f.readlines()),
    'lines'
    f.close()
```

Handling Exceptions, Example

```
numbers = [3.7832, 8.5, 1.9, 0]
for x in numbers:
    print x,
    try:
        print 1.0 / x
    except ZeroDivisionError:
        print '*** has no inverse element ***'
```

```
>>> execfile("exception.py")
3.7832 0.264326496088
8.5 0.117647058824
1.9 0.526315789474
0 *** has no inverse element ***
```

Multiple Exceptions

```
import sys

try:
    f = open('integers.txt')
    s = f.readline()
    i = int(s.strip())

except IOError as (errno, strerror):
    print "I/O error({0}): {1}".format(errno,
    strerror)

except ValueError:
    print "No valid integer in line."

except:
    print "Unexpected error:", sys.exc_info()[0]
    raise
```

Python II

except with Multiple Exceptions

An except clause may name more than one exception in a tuple of error names, as we see in the following example:

```
try:
    f = open('integers.txt')
    s = f.readline()
    i = int(s.strip())
except (IOError, ValueError):
    print "An I/O error or a ValueError
    occurred"
except:
    print "An unexpected error occurred"
    raise
```

Clean-up Actions

So far the try statement had always been paired with except clauses.

But there is another way to use it as well.

The try statement can be followed by a finally clause.

Finally clauses are called clean-up or termination clauses, because they must be executed under all circumstances, regardless if an exception occurred in a try block or not.

Raising Exceptions

The raise statement allows the programmer to raise an exception if an error occurs:

```
>>> raise SyntaxError("Text of error message")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
SyntaxError: Text of error message
```

Example: try ... finally

```
try:
    x = float(raw_input("Your number: "))
    inverse = 1.0 / x
finally:
    print("There may or may not have been an
    exception.")
    print "The inverse: ", inverse
```

the output of the previous script:

```
bernd@venus:~/tmp$ python finally.py
Your number: 34
There may or may not have been an exception.
The inverse: 0.0294117647059
bernd@venus:~/tmp$ python finally.py
Your number: Python
There may or may not have been an exception.
Traceback (most recent call last):
  File "finally.py", line 3, in <module>
    x = float(raw_input("Your number: "))
ValueError: invalid literal for float(): Python
bernd@venus:~/tmp$
```

Python II

Combining try, except and finally

```
try:
    x = float(raw_input("Your number: "))
    inverse = 1.0 / x
except ValueError:
    print "Only int or float are possible"
except ZeroDivisionError:
    print "Infinity"
finally:
    print("There may or may not have been an exception.")
```

Using assert

```
>>> x = 5
>>> y = 3
>>> assert x < y, "x has to be smaller than y"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AssertionError: x has to be smaller than y
>>>
```

assert should not be used to catch programming errors like $x / 0$, because Python traps such programming errors itself!

assert is for trapping user-defined constraints!

The assert Statement

The assert statement is intended for debugging statements. It can be seen as an abbreviated notation for a conditional raise statement:

```
if not <some_test>:
    raise AssertionError(<message>)
```

This can be written as:

```
assert <some_test>, <message>
```

In other words:

If `<some_test>` evaluates to False, an exception is raised and `<message>` will be output.

Push/Append and Pop

An Array can be looked at as a stack of Information. New elements are put on top of the stack, or the highest index of the array, („right side of the array“)



In Perl, these functions are called `pop()` and `push()`.

In Python: `pop()` and `append()`

`s.append(x)`

x will be appended to the end of the list
the i^{th} element of s will be returned and removed from the stack s.

`s.pop(i)`

`s.pop()`

If `pop()` is called with no argument, the top element will be removed and returned.

Python II

Example

```
>>> colors=["red","green"]
>>> colors.append("blue")
>>> colors
['red', 'green', 'blue']
>>> c = colors.pop()
>>> c
'blue'
>>> colors
['red', 'green']
>>> colors.append(colors.pop())
>>> colors
['red', 'green']
```

extend()

The method `extend` can be used to append more than one element to the list.

`s.extend(t)`
`t` has to be an iterable object

```
>>> fib = [0,1,1,2,3,5]
>>> fib.extend([8,13,21])
>>> fib
[0, 1, 1, 2, 3, 5, 8, 13, 21]
Take care of the difference to append:
>>> fib = [0,1,1,2,3,5]
>>> fib.append([8,13,21])
>>> fib
[0, 1, 1, 2, 3, 5, [8, 13, 21]]
```

Reversing a List

```
def rev(lst):
    revl = []
    while (lst):
        el = lst.pop()
        revl.append(el)
    return revl

list = ["a","b","c","d"]
list = rev(list)
print list
```

The function `rev()` returns a list in reverse sequence:

```
$ python reverse.py
['d', 'c', 'b', 'a']
```

extend()

`extend()` can also take a tuple as an argument. The result remains the same:

```
>>> t = (8,13,21)
>>> l = [8,13,21]
>>> fib = [0,1,1,2,3,5]
>>> fib.extend(t)
>>> print fib
[0, 1, 1, 2, 3, 5, 8, 13, 21]
>>> fib = [0,1,1,2,3,5]
>>> fib.extend(l)
>>> print fib
[0, 1, 1, 2, 3, 5, 8, 13, 21]
>>>
```

Python II

Exercise

Write a function `flatten()` using the methods `extend()` and `append()`, which turns an arbitrary nested list (tuple) into a flat list.
hint: `type().__name__` will also be needed.



```
>>> flatten([(1,2), "Python", ["a",1], 1, 1.3])
[1, 2, 'Python', 'a', 1, 1, 1.3]
```

Removing Values

It's possible to remove elements with the method `remove()` without knowing the corresponding indices.

`s.remove(x)`

This call removes the first occurrence of the value `x` from the list `s`.
If `x` is not contained in the list, the following exception is thrown:

`„ValueError: list.remove(x): x not in list“`

```
>>> colors=["red", "green", "blue", "green", "yellow"]
>>> colors.remove("green")
>>> colors
['red', 'blue', 'green', 'yellow']
```

Solution: `flatten()`

```
def flatten(x):
    """flatten(sequence) -> list"""

    result = []
    for el in x:
        # alternatively:
        # if isinstance(el, (list, tuple)):
        if (type(el) == list) or (type(el)
        == tuple):
            result.extend(flatten(el))
        else:
            result.append(el)
    return result
```

Contained in List

It's possible to test with `el in list`, if „`el`“ is contained in the list `list`. The result is `True`, if „`el`“ is contained once or multiple times in the list.

If you are interested in the exact number, you can use the method `count()`.

```
>>> colors=['blue', 'green', 'red', 'green']
>>> 'green' in colors
True
>>> colors.count('green')
2
>>> colors.count('black')
0
```

Python II

index()

The position of an element inside of a list can be determined with the methode **index()**

```
s.index(x[, i[, j]])
```

The optional parameter *i* sets a start index and *j* a end position, i.e. *x* will be searched for between the position *i* and *j*.

```
>>> colors=['blue', 'green', 'red', 'green']
>>> colors.index('green')
1
>>> colors.index('green',2)
3
>>> colors.index('green',2,4)
3
```

insert and append

```
>>> abc = ["a","b","c"]
>>> abc.insert(len(abc),"d")
>>> print abc
['a', 'b', 'c', 'd']
>>>
```

is equivalent to:

```
>>> abc = ["a","b","c"]
>>> abc.append("d")
>>> print abc
['a', 'b', 'c', 'd']
>>>
```

Finding the Indices of all Matching Items

```
l = ["a", "b", "c","a","b","a","x"]
i = -1
while "a" in l[i+1:]:
    i = l.index("a",i+1)
    print i
```

Reverting an Array



```
>>> simpsons =
["Lisa", "Homer", "Bart", "Maggie", "Marge"]
>>> simpsons.reverse()
>>> simpsons
['Marge', 'Maggie', 'Bart', 'Homer',
'Lisa']
```

Python II

sort

Python lists have a built-in `sort()` method that modifies the list in-place and a `sorted()` built-in function that builds a new sorted list from an iterable.



```
>>> lst = [123, 17, 4, 69, 9, 259, 99]
>>> print sorted(lst)
[4, 9, 17, 69, 99, 123, 259]
>>> print lst
[123, 17, 4, 69, 9, 259, 99]
>>> lst.sort()
>>> print lst
[4, 9, 17, 69, 99, 123, 259]
>>>
```

sort with Key Function

Both `list.sort()` and `sorted()` have an optional `key` parameter to specify a function to be called on each list element prior to making comparisons.

```
>>> lst =
["Apple", "House", "cars", "Cats", "Red", "all"]
>>> sorted(lst)
['Apple', 'Cats', 'House', 'Red', 'all',
'cars']
>>> sorted(lst, key=str.lower)
['all', 'Apple', 'cars', 'Cats', 'House',
'Red']
```

Defining an Ordering

Task: Sort a list of numbers as strings (alphabetically)

```
def sort_as_strings(x,y):
    if (str(x) < str(y)):
        return -1
    elif (str(x) > str(y)):
        return 1
    else:
        return 0
lst = [23, 2, 4, 12, 1, 123, 243, 3, 33]
lst.sort(cmp=sort_as_strings)
print lst
The result of the call:
$ python sort_as_strings.py
[1, 12, 123, 2, 23, 243, 3, 33, 4]
```

Sorting Complex Numbers

To be able to order a list using `sort()` or `sorted()`, there has to exist a partial order on the elements of the list.

There is no total order on the complex numbers:

```
>>> lst = [1+1j, 2+3j, 4+1j, -4-1j]
>>> lst.sort()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: no ordering relation is defined for
complex numbers
>
```

Python II

bodenseo

Python 2

Order on Complex Numbers

Not satisfying, but at least an ordering:

```
>>> complex = [1 + 0j, 10 + 0j, 2 + 0j, 2 + 1j]
>>> sorted(complex)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: no ordering relation is defined for
complex numbers
>>> sorted(complex, key=str)
[(1+0j), (10+0j), (2+0j), (2+1j)]
```

The complex numbers are turned into strings before ordering them.

bodenseo, Bernd Klein, 09/04/13

Folie 45

bodenseo

Python 2

Parameter cmp

$l = [l_1, l_2 \dots l_n]$

$l_1, l_2 \dots l_n$ are of data type T , for which we want to define an ordering.

A function f , taking two arguments of data type T and returning an integer value, can be used for sorting with `cmp`.

Two neighbouring list elements l_i and l_{i+1} will be swapped, if $f(l_i, l_{i+1}) > 0$.

The Python call is:

```
l.sort(f)
```

bodenseo, Bernd Klein, 09/04/13

Folie 47

bodenseo

Python 2

Complex Numbers: Alternative

```
def complex_compare(x):
    xl = abs(x)
    return xl

lst = [1+1j, 0+0j, 4+1j, 2+3j, -4-1j]

lst.sort(key=complex_compare)

print "sorted List: ", lst
```

bodenseo, Bernd Klein, 09/04/13

Folie 46

bodenseo

Python 2

Parameter key

Let l again be a list of data type T , on which we define an order:

$l = [l_1, l_2 \dots l_n]$

Let f be a function with parameters of data type T . f is a mapping into a data type, where there is an ordering in Python available.

f can be used for sorting with the `key`

The list $[f(l_1), f(l_2) \dots f(l_n)]$ is sorted and the original list l will be sorted correspondingly, i.e. neighbouring list elements l_i and l_{i+1} will be swapped if

$f(l_i) > f(l_{i+1})$.

The correct Python call is:

```
l.sort(key=f)
```

bodenseo, Bernd Klein, 09/04/13

Folie 48

Python II

Reverse Sort

If the ordering shall be reversed, the third parameter reverse has to be set to True.

```
>>> colors
['red', 'black', 'blue', 'green', 'white',
'yellow']
>>> colors.sort(reverse=True)
>>> colors
['yellow', 'white', 'red', 'green', 'blue',
'black']
>>> colors.sort()
>>> colors
['black', 'blue', 'green', 'red', 'white',
'yellow']
```

And Another One

```
>>> l = [("Peter",114,3467.87),
("Eve",117,3100.03),
("Bill",105,7239.34),
("Stella",125,3983.45)]
>>> l.sort(key=lambda x: x[0])
>>> l
[('Bill', 105, 7239.3400000000001),
('Eve', 117, 3100.0300000000002),
('Peter', 114, 3467.8699999999999),
('Stella', 125, 3983.4499999999998)]
>>> l.sort(key=lambda x: x[1])
>>> l
[('Bill', 105, 7239.3400000000001),
('Peter', 114, 3467.8699999999999),
('Eve', 117, 3100.0300000000002),
('Stella', 125, 3983.4499999999998)]
```

Another Example

Sorting of a list, which consists of 2-Tuples, by sorting them according to the second tuple element.

```
>>> lst=[[2,6],[1,3],[5,4],[1,0]]
>>> lst.sort(key=lambda x: x[1])
>>> lst
[[1, 0], [1, 3], [5, 4], [2, 6]]
```

Sort according to the sum of the components of a 2-tuple:

```
>>> lst=[[2,6],[1,3],[5,4],[1,0]]
>>> lst.sort(key = lambda x: x[0] + x[1])
>>> print(lst)
[[1, 0], [1, 3], [2, 6], [5, 4]]
>>>
```

Exercise

Write a function, which determines the frequency of letters of a text.

A sorted (frequency) list with the letters and their frequency shall be returned.

Special characters and digits shouldn't be counted.

Example:

```
string = "Monty Python"
x = letter_frequency(string)
print x
```

Output:

```
[('o', 2), ('n', 2), ('t', 2), ('y', 2),
('h', 1), ('m', 1), ('p', 1)]
```

Python II

Solution

```
#!/usr/bin/env python
# -*- coding: iso-8859-15 -*-

def letter_frequency(s):
    frequency_dict = {}
    for char in s.lower():
        if char.isalpha():
            if char in frequency_dict:
                frequency_dict[char] += 1
            else:
                frequency_dict[char] = 1
    f = frequency_dict.items()
    f.sort(key = lambda x: x[1], reverse=True)
    return f

freq = letter_frequency(open("1984.txt").read())
print freq
```

Tuples with a Single Element

```
>>> single = (1)
>>> type(single)
<type 'int'>
>>> single = (1,)
>>> type(single)
<type 'tuple'>
>>> single
(1,)
```

It doesn't work with (1), because this is not a tuple instance but an expression, i.e. a 1 in parenthesis.

Immutable Lists: Tuple

Tuples are created with (round) brackets:

```
>>> t = (0,1,1,2,3,5,8)
>>> t[3]
2
```



Tuples support only the fundamental functions of the list data type, i.e. the functions which are not changing a tuple.

An empty tuple can be defined with empty brackets ():
e = ();

Tuple packing und unpacking

```
>>> date = 31,12,2008  ← packing
>>> date
(31, 12, 2008)
>>> (day,month,year) = date ← unpacking
>>> day
31
>>> month
12
>>> year
2008
```

This can be used to swap two variables:

```
>>> x, y = 47, 11
>>> x, y = y, x
```

Python II

Changeable Unchangeables

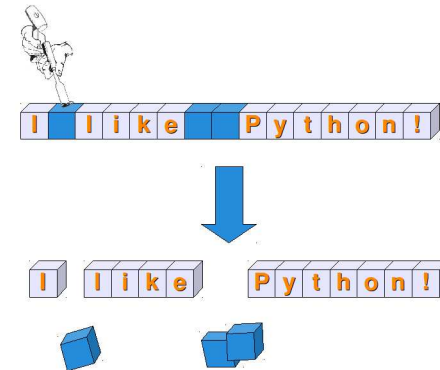
And yet it ~~moves~~ changes ...

```
>>> galileo = ([],)
>>> galileo[0].append("And yet it moves!")
>>> galileo
(['And yet it moves!'],)
```

But this isn't possible:

```
>>> galileo[0] = [3434,3434]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support
item assignment
```

Splitting Strings



Close to all about Strings



Disassembling Strings

Strings can be disassembled with the following strings:

- `s.split([sep[, maxsplit]])`
- `s.rsplit([sep[, maxsplit]])`
- `s.splitlines([keepends])`
- `s.partition(sep)`
- `s.rpartition(sep)`

`sep` stands for the delimiter string.

If `sep` is not specified or `None`, any whitespace string is a separator.

`maxsplit` defines the maximal number of splittings.

Python II

Examples

```
>>> mammon = "The god of the world's leading religion. The chief temple is in the holy city of New York."
>>> mammon.split()
['The', 'god', 'of', 'the', "world's", 'leading', 'religion.', 'The', 'chief', 'temple', 'is', 'in', 'the', 'holy', 'city', 'of', 'New', 'York.']
>>> mammon.split(".")
["The god of the world's leading religion", 'The chief temple is in the holy city of New York', '']
>>> mammon.split(" ", 4)
['The', 'god', 'of', 'the', "world's leading religion. The chief temple is in the holy city of New York."]
```

More on split

If sep is given, consecutive delimiters are not grouped together and are deemed to delimit empty strings (e.g. '1,,2'.split(',') returns ['1', '', '2']). The sep argument may consist of multiple characters (e.g., '1<>2<>3'.split('<>') returns ['1', '2', '3']). Splitting an empty string with a specified separator returns [].

```
>>> doc = "/usr/share/doc/python/html"
>>> doc.split("/")
['', 'usr', 'share', 'doc', 'python', '', 'html']
```

rsplit

rsplit behaves like **split**, but splits from the right:

```
>>> mammon = "The god of the world's leading religion. The chief temple is in the holy city of New York."
>>> mammon.rsplit()
['The', 'god', 'of', 'the', "world's", 'leading', 'religion.', 'The', 'chief', 'temple', 'is', 'in', 'the', 'holy', 'city', 'of', 'New', 'York.']
>>> mammon.rsplit(None, 4)
["The god of the world's leading religion. The chief temple is in the holy", 'city', 'of', 'New', 'York.']
>>>
```

Special case with split()

If sep is not specified or is None, a different splitting algorithm is applied: runs of consecutive whitespace are regarded as a single separator, and the result will contain no empty strings at the start or end if the string has leading or trailing whitespace. Consequently, splitting an empty string or a string consisting of just whitespace with a None separator returns [].

```
>>> mammon = "The  god  of the world's">>>
mammon.split()
['The', 'god', 'of', 'the', "world's"]
>>> mammon.split(" ")
['The', '', '', 'god', '', '', 'of', 'the', "world's"]
>>>
```

Python II

Disassemble String in Lines

str.splitlines([keepends])

Returns a list of the lines in the string, breaking at line boundaries. Line breaks are not included in the resulting list unless keepends is given and true.

```
>>> rhyme = "An apple a day\nSends the doctor away\n"
>>> rhyme.splitlines()
['An apple a day', 'Sends the doctor away']
>>> rhyme.splitlines(True)
['An apple a day\n', 'Sends the doctor away\n']
By comparison:
>>> rhyme.split("\n")
['An apple a day', 'Sends the doctor away', '']
>>> rhyme.split()
['An', 'apple', 'a', 'day', 'Sends', 'the', 'doctor', 'away']
```

join()

str.join(iterable)

Returns a string which is the concatenation of the strings in the iterable **iterable**.

The separator between elements is the string providing this method.

Examples:

```
>>> x = ["Python", "Perl", "Java"]
>>> "-".join(x)
'Python-Perl-Java'
>>> x = "48772"
>>> ".".join(x)
'4.8.7.7.2'
```

Partition

str.partition(sep)

Split the string at the first occurrence of sep, and return a 3-tuple containing the part before the separator, the separator itself, and the part after the separator.

If the separator is not found, return a 3-tuple containing the string itself, followed by two empty strings.

```
>>> spruch = "Eine Katze, die jagt, hungert nicht!"
>>> spruch.partition(",")
('Eine Katze', ',', ' die jagt, hungert nicht!')
>>> spruch.rpartition(",")
('Eine Katze, die jagt', ',', ' hungert nicht!')
```

Finding Substring

The following methods can be used to search for substrings:

- **s.find(substring[, start[, end]])**
- **s.rfind(substring[, start[, end]])**
- **s.index(substring[, start[, end]])**
- **s.rindex(substring[, start[, end]])**
- **s.count(substring[, start[, end]])**

Python II

Explanatory Examples

```
>>> rhyme = 'Little Jack Horner sat in a  
corner,\nEating a Christmas pie;\nHe put in his thumb  
and pulled out a plum,\nAnd said, "What a good boy am  
I!" '  
>>> rhyme.find("orner")  
13  
>>> rhyme.find("orner", 14)  
29  
>>> rhyme.count("orner")  
2  
>>> rhyme.index("orner")  
13  
>>> rhyme.rindex("orner")  
29  
>>> rhyme.rindex("girl")  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
ValueError: substring not found  
>>> rhyme.find("girl")  
-1
```

String Substitution

To substitute a substring **old** by a substring **new** we can use the method `replace`:

```
s.replace(old, new[, count])
```

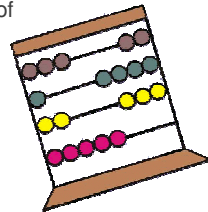
Without using the optional parameter **count** every occurrence of **old** will be replaced by **new**.

The optional parameter **count** can be used to control how many occurrences of **old** will be replaced by **new**.

Exercise

Write a function which calculates the position of the *n*-th occurrence of a string *sub* in another string. If *sub* doesn't occur, -1 shall be returned.

```
def findnth(str, substr, n):  
    num = 0  
    start = -1  
    while num < n:  
        start = str.find(substr, start+1)  
        if start == -1: return -1  
        num += 1  
  
    return start
```



Examples

```
>>> ch = "Drei Chinesen mit dem Kontrabass"  
>>> ch.replace("e", "a")  
'Drai Chinasan mit dam Kontrabass'  
>>> ch.replace("e", "a").replace("i", "a")  
'Draa Chanasan mat dam Kontrabass'  
>>>  
ch.replace("e", "a").replace("i", "a").replace("o",  
    "a")  
'Draa Chanasan mat dam Kantrabass'  
>>>
```

Python II

Exercise

Write a function which replaces the n-th occurrence of a string **sub** by a string **replacement**.

The function shall return the changed string or the unchanged, or the original string, if **replacement** didn't occur in the string.

Some more Methods

```
>>> txt.lower()
'drei chinesen mit dem kontrabass'
>>> txt.upper()
'DREI CHINESEN MIT DEM KONTRABASS'
>>> txt.swapcase()
'dREI cHINESEN MIT DEM kONTRABASS'
>>> txt.capitalize()
'Drei chinesen mit dem kontrabass'
>>> txt
'Drei Chinesen mit dem Kontrabass'
>>> txt.title()
'Drei Chinesen Mit Dem Kontrabass'
```

Sample Solution

```
def findnth(str, substr, n):
    num = 0
    start = -1
    while num < n:
        start = str.find(substr, start+1)
        if start == -1: return -1
        num += 1

    return start

def replacenth(source, search, replacement, n):
    pos = findnth(source, search, n)
    if pos == -1: return source
    return source[:pos] + replacement + \
    source[pos+len(search):]
```

Exercise: Morse Code II

A	• —	U	• • —
B	• • • —	V	• • • •
C	• — • —	W	• — • —
D	• — • —	X	• • — •
E	•	Y	• — • •
F	• • — •	Z	• — • —
G	• — • —		
H	• • • •		
I	• •		
J	• — • — • —		
K	• — • —	1	• — • — • —
L	• — • •	2	• • • — • —
M	• — • —	3	• • • • — •
N	• — • —	4	• • • • — •
O	• — • —	5	• • • • — •
P	• — • •	6	• • • • — •
Q	• — • •	7	• • • • — •
R	• — • •	8	• • • • — •
S	• • •	9	• • • • — •
T	• —	0	• • • • — •

Write a function which translates a string in Morse code into a „normal“ string.

The Morse character are separated by spaces. Words by three spaces.

Python II

Stripping Strings

Sometimes strings have trailing or leading characters which have to be removed. In most cases whitespaces.

- **`s.strip([chars])`**
Return a copy of the string with the leading and trailing characters removed. The chars argument is a string specifying the set of characters to be removed. If omitted or None, the chars argument defaults to removing whitespace. The chars argument is not a prefix or suffix; rather, all combinations of its values are stripped
- **`s.lstrip([chars])`**
like strip(), but only from left side.
- **`s.rstrip([chars])`**
like strip() but from right side.

Centering Strings

- **`str.center(width[, fillchar])`**
Return a centered string of length width. Padding is done using the specified fillchar (default is a space). The original string is returned if width is less than `len(str)`
- **`str.ljust(width[, fillchar])`**
like center, but the padding is done from the left side
- **`str.rjust(width[, fillchar])`**
like ljust, but from the right side
- **`str.zfill(width)`**
Return the numeric string left filled with zeros in a string of length width. A sign prefix is handled correctly. The original string is returned if width is less than `len(s)`.

Examples

```
>>> str = "\n\n The quick brown fox jumps over  
the lazy dog\t \r\n"  
>>> print str
```

```
    The quick brown fox jumps over the lazy dog
```

```
>>> str = str.strip()  
>>> print str  
The quick brown fox jumps over the lazy dog  
>>>
```

String Tests

- `s.isalnum()`** True, if the string is entirely composed of alphabetic and/or numeric characters
- `s.isalpha()`** True, if all characters in s are alphabetic
- `s.isdigit()`** True, if all character in s are digits
- `s.islower()`** True, if all character in s are lower case letters
- `s.isupper()`** True, if all character in s are upper case letters
- `s.isspace()`** True, if all characters in s are whitespaces
- `s.istitle()`** True, if all words in s are capitalized

Python II

What's a String?

Are strings just arrays of character elements?

A string is an ordered set of character depending on the representation in the Computer.

Examples:

What is a string?

Что такое комбинация?

ما هو وتر؟

什麼是弦？

Unicode-Encodings

Unicode is a specification, which assigns a unique number to any (known) character from all over the world.

An encoding is a mapping, which assigns a computer representation in Bytes to these unique numbers.

We need an Encoding to save a Unicode string.

There are different encodings LATIN-1, UTF-16LE, UTF-8

Code Table

A Code Table can be seen as a mapping from the internal computer representation into the screen representation.

Code	...0	...1	...2	...3	...4	...5	...6	...7	...8	...9	...A	...B	...C	...D	...E	...F
0...	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1...	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2...	SP	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3...	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4...	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5...	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6...	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7...	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL

Unicode

ЮНИКОД
(Russian)

یونی‌کد
(Persian)

统一码

இருங்குறி (Tamil)

Հայկական
(Armenian)

یکرمزی
(Urdu)

يونيكود
(Arabic)

यूनि‌कोड
(Hindi)

ਯੂਨੀਕੋਡ
(Punjabi)

Python II

UTF-16LE and UTF-8

UTF-16LE:

A character encoding that maps code points of Unicode character set to a sequence of 2 bytes (16 bits). UTF-16LE stands for Unicode Transformation Format - 16-bit Little Endian.

UTF-8

stands for Unicode Transformation Format-8. It is an octet (8-bit) lossless encoding of Unicode characters.

UTF-8 encodes each Unicode character as a variable number of 1 to 4 octets, where the number of octets depends on the integer value assigned to the Unicode character.

Encoding-Cookie

Example of an encoding cookie, which has to be placed in the first or the second line of the file.

```
# -*- coding: iso-8859-1 -*- or
# -*- coding: utf-8 -*-
```

The default encoding:

```
>>> sys.getdefaultencoding()
'ascii'
```

The Encoding-Cookies has to match the Encoding of the file, i.e. the file for the encoding above has to be iso-8859-1.

As soon as the encoding is set correctly, special characters like "Umlaute" can be used, if they are supported by the chosen encoding.

Umlaute in Python

Python uses two different types of objects for strings:
str and **unicode**

If y text without any special declarations is written in quotes (e.g. `txt = "an example"`), an object of the type `str` will be created.

The type `str` consists of characters of the English language, i.e. no Umlaute.

To use Umlaute, we need a special Encoding-Cookies.

encode and decode

```
>>> s = "Hello"
>>> b = s.encode()
>>> type(b)
<class 'bytes'>
>>> ba = bytearray(s)
Traceback (most recent call last):
  File "<stdin>", line 1, in
<module>
TypeError: string argument without
an encoding
>>> ba = bytearray(b)
>>> ba[1] = 97
>>> print(ba)
bytearray(b'Hallo')
>>> type(b.decode())
<class 'str'>
>>> type(b.decode().encode())
<class 'bytes'>
```

Python II

Example with Umlauten

```
# -*- coding: iso-8859-1 -*-  
  
str = "Mädchen mögen Hüte."  
  
# Convert Text in Unicode  
str_unicode = str.decode("iso-8859-1")  
  
# Encode Text into UTF-8  
str_utf8 = str_unicode.encode("utf-8")  
  
print str  
print str_unicode  
print str_utf8
```

Modular Programming

Modular programming is a software design technique that increases the extent to which software is composed of separate parts, called modules.

With Modular Programming, concerns are separated such that no (or few) modules depend upon other modules of the system.

Minimization of dependencies is the goal.

When creating a modular system, instead of creating a monolithic application, several smaller modules are built separately that, when composed together, will construct the executable application program.



Regular Expressions



Python Modules

A module is a file containing Python definitions and statements. The file name is the module name with the suffix .py appended.

The following is saved in fibo.py:

```
def fib(n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fib(n-1) + fib(n-2)  
  
def ifib(n):  
    a, b = 0, 1  
    for i in range(n):  
        a, b = b, a + b  
    return a
```

Python II

Applying the Previous Example

```
>>> import fibo
>>> fibo.fib(30)
832040
>>> fibo.ifib(100)
354224848179261915075L
>>> fibo.ifib(1000)
43466557686937456435688527675040625802564660517
37178040248172908953655541794905189040387984007
92551692959225930803226347752096896232398733224
71161642996440906533187938298969649928516003704
476137795166849228875L
>>>
A local name can be assigned to a module function:
>>> fib = fibo.ifib
>>> fib(10)
55
>>>
```

Importing Names Directly

Names from a module can directly be imported into the importing module's symbol table:

```
>>> from fibo import fib, ifib
>>> ifib(500)
1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

This does not introduce the module name from which the imports are taken in the local symbol table.

Import all names defined in a module:

```
>>> from fibo import *
>>> fib(500)
1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

This imports all names except those beginning with an underscore (_).

More on Modules

Usually, modules contain functions, but there can be statements as well. These statements are used to initialize the module. They are only executed once, when the module is imported.

A module has a private symbol table, which is used as the global symbol table by all functions defined in the module.

This way global variables of a module can't accidentally clash with a user's global variables.

Global variables of a module can be accessed with the same notation as functions, i.e.

```
modname.name
```

A module can import other modules. It is customary to place all import statements at the beginning of a module/script.

Executing Modules as Scripts

A Python module can be run like a script:

```
python fibo.py <arguments>
```

In this case `__name__` is set to `"__main__"`.

With the following conditional statement the file can be used as a module or as a script:

```
if __name__ == "__main__":
    import sys
    fib(int(sys.argv[1]))
```

The code that parses the command line will only run if the module is executed as the "main" file:

```
$ python fibo.py 50
1 1 2 3 5 8 13 21 34
```

If the module is imported, the code is not run:

```
>>> import fibo
>>>
```

Python II

Renaming a Namespace

While importing a module, the name of the namespace can be changed:

```
>>> import math as mathematics
>>> print mathematics.cos(mathematics.pi)
-1.0
```

After this import there exists a namespace `mathematics` but no namespace `math`.

```
>>> from math import pi, pow as power, sin as
sinus
>>> power(2,3)
8.0
>>> sinus(pi)
1.2246467991473532e-16
```

The Module Search Path

Python's module search path consists of the following directories:

- 1) The directory of the top-level file, i.e. the file being executed.
- 2) `PYTHONPATH` directories, if set
- 3) standard installation path Linux/Unix e.g. in `/usr/lib/python2.5`.

Finding out where a module is located:

```
>>> import math
>>> math.__file__
'/usr/lib/python2.5/lib-dynload/math.so'
>>> import random
>>> random.__file__
'/usr/lib/python2.5/random.pyc'
```

Kinds of Modules

There are different kind of modules:

- 1) written in Python
suffix: `.py`
- 2) Dynamically linked C modules
suffixes: `.dll`, `.pyd`, `.so`, `.sl`, ...
- 3) C-Modules, linked with the interpreter:
to get a list of these modules:

```
import sys
print sys.builtin_module_names
```


An error message is returned for Built-in-Modules.

Directories with Modules

```
>>> import sys
>>> for dir in sys.path:
...     print(dir)
...
```



```
/usr/lib/python3.2
/usr/lib/python3.2/plat-linux2
/usr/lib/python3.2/lib-dynload
/usr/local/lib/python3.2/dist-packages
/usr/lib/python3/dist-packages
```

Python II

dir() Function

With an argument, returns a list of valid attributes for that object.

```
>>> dir(math)
['__doc__', '__file__', '__name__', 'acos',
'asin', 'atan', 'atan2', 'ceil', 'cos', 'cosh',
'degrees', 'e', 'exp', 'fabs', 'floor', 'fmod',
'frexp', 'hypot', 'ldexp', 'log', 'log10', 'modf',
'pi', 'pow', 'radians', 'sin', 'sinh', 'sqrt',
'tan', 'tanh']
>>>
Without arguments, return the list of names in the current local scope.
>>> import math
>>> col = ["red", "green", "blue"]
>>> dir()
['__builtins__', '__doc__', '__name__', 'col',
'math']
```

List of built-in-Functions

```
>>> import __builtin__
>>> dir(__builtin__)
['ArithmeticError', 'AssertionError',
'AttributeError', 'BaseException',
'DeprecationWarning', 'EOFError',
'Ellipsis', 'EnvironmentError',
'Exception', 'False', 'FloatingPointError',
'FutureWarning', 'GeneratorExit',
'IOError', 'ImportError', 'ImportWarning',
'IndentationError', 'IndexError',
'KeyError', 'KeyboardInterrupt',
'LookupError', 'MemoryError', 'NameError',
'None', 'NotImplemented',
'NotImplementedError', 'OSError',
'OverflowError',
'PendingDeprecationWarning',
...]
```

dir and help

You can get additional information on a module with help :

```
jupyter $ python
Python 2.5.2 (r252:60911, Oct  5 2008,
19:29:17)
[GCC 4.3.2] on linux2
Type "help", "copyright", "credits" or
"license" for more information.
>>> import math
>>> help(math)
```

The os Module

The os-Module provides an generic interface to the operating system's basic tools.

<code>os.chdir(path)</code>	change working directory
<code>os.listdir()</code>	returns a list of all the files in the specified directory
<code>os.getcwd()</code>	returns a string referring to the current working directory
<code>os.rename(src, dst)</code>	renames src to dst
<code>os.remove(file)</code>	removes file
<code>os.removedirs(path)</code>	like rmdir() but recursively
<code>os.system(command)</code>	Executes the shell command in a subshell; the return code of the command is returned

Python II

Module Packages

It's possible to put several modules into a Package.
A directory of Python code is said to be a package.

A package is imported like a "normal" module.

Each directory named within the path of a package must also contain a file named `__init__.py`, or else your package import will fail.



Simple Example

a.py contains:

```
def f1():  
    print("Hello, f1 from module 'a' calling")
```

b.py contains:

```
def foo():  
    print("Hello, foo from module 'b' calling")
```

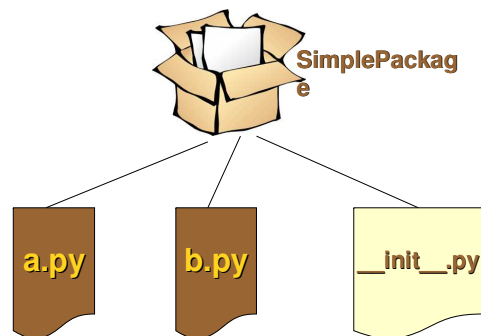
Using the package „SimplePackage:

```
from SimplePackage import a,b  
a.f1()  
b.foo()
```

Output:

```
Hello, f1 from module 'a' calling  
Hello, foo from module 'b' calling
```

Simple Example



Generators

It is possible to write functions that may be resumed after they send a value back.

These functions are called generators because they generate a sequence of values over time.

The main difference between functions and generators is the generators yield a value instead of returning it, i.e. a **yield** statement is used instead of a **return** statement



Python II

Iterators in for Loops

```
>>> cities = ["Paris", "Berlin", "London"]
>>> for city in cities:
...     print("city: " + city)
...
city: Paris
city: Berlin
city: London
```

Whats happening?

- The built-in iter with iter(cities), which calls `__iter__` of the list class. The return value – a „list_iterator“ - is stored in a variable, e.g. `iter_x`.
- After each loop the next function ist called with `iter_x` as an argument and the return value of this call is assigned to the variable `iter_x`.
- The loop will be repeated until `next(iter_x)` raises the exception `StopIteration`.

Simple Example

```
>>> def abc_generator():
...     yield("a")
...     yield("b")
...     yield("c")
...
>>> for i in abc_generator():
...     print i
...
a
b
c
>>>
```

Generators are as easy as the



Iterating manually

```
>>> l = [23, 34, 56, 78]
>>> iter_x = iter(l)
>>> next(iter_x)
23
>>> next(iter_x)
34
>>> next(iter_x)
56
>>> next(iter_x)
78
>>> next(iter_x)
Traceback (most recent
call last):
File "<stdin>", line
1, in <module>
StopIteration
>>>
```

PI with Generators

The following sequence converges to $\pi/4$

$$\pi/4 = 1 - 1/3 + 1/5 - 1/7 + \dots$$

The implementation as a generator:

```
def pi_series():
    sum = 0
    i = 1.0; j = 1
    while True:
        sum = sum + j/i
        yield 4*sum
        i += 2
        j *= -1
    for i in pi_series():
        print i
```

Exercise:

The program has to be stopped with Ctrl-C. Add an exit condition!

Python II

bodenseo

Python 2

Elegant method with next()

```
def firstn(g, n):
    for i in range(n):
        yield g.next()
```

```
print list(firstn(pi_series(), 8))
```

A call returns:

```
$ python pi_generator.py
[4.0, 2.666666666666667, 3.466666666666667,
2.8952380952380956, 3.3396825396825403,
2.9760461760461765, 3.2837384837384844,
3.0170718170718178]
$
```

bodenseo, Bernd Klein, 09/04/13

Folie: 113

bodenseo

Python 2

Send-Method, Python3

```
def abc():
    s = "abcdef"
    count = 0
    while True:
        i = (yield s[count %
6])
        if i:
            count = i
        else:
            count += 1

if __name__ == "__main__":
    x = abc()
    print(next(x))
    print(next(x))
    print(x.send(4))
    print(next(x))
```

Output:

a
b
e
f

bodenseo, Bernd Klein, 09/04/13

Folie: 115

bodenseo

Python 2

Another Example

```
>>> def abc():
...     while True:
...         yield("a")
...         yield("b")
...         yield("c")
...         yield("d")
...         yield("e")
...
>>> def firstn(g,n):
...     for i in range(n):
...         yield g.next()
...
>>>
```

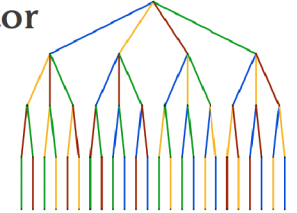
bodenseo, Bernd Klein, 09/04/13

Folie: 114

bodenseo

Python 2

Permutations Generator



```
def permutations(items):
    n = len(items)
    if n==0: yield []
    else:
        for i in range(len(items)):
            for cc in permutations(items[:i]
+items[i+1:]):
                yield [items[i]]+cc

for p in permutations(['r','e','d']): print ''.join(p)

for p in permutations(list("game")): print ''.join(p)
```

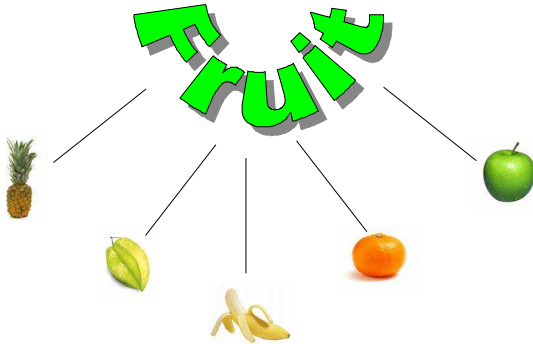
bodenseo, Bernd Klein, 09/04/13

Folie: 116

Python II

Python II

Objects and Classes



Object Oriented Programming

Object-oriented programming (OOP) is a programming paradigm that uses "objects" and their interactions to design applications and computer programs.

It is based on several techniques:

- **encapsulation**
- **modularity**
- **polymorphism**
- **inheritance**
- **abstraction**

Criticism

Edsger W. Dijkstra wrote: "... what society overwhelmingly asks for is snake oil. Of course, the snake oil has the most impressive names —otherwise you would be selling nothing— like "Structured Analysis and Design", "Software Engineering", "Maturity Models", "Management Information Systems", "Integrated Project Support Environments" "Object Orientation" and "Business Process Re-engineering" (the latter three being known as IPSE, OO and BPR, respectively)."

Alexander Stepanov suggested that OOP provides a mathematically-limited viewpoint and called it, "almost as much of a hoax as Artificial Intelligence"

"I used OOP when working on the Lisp Machine window systems, and I disagree with the usual view that it is a superior way to program." (Richard Stallman, 1995)

Strawberry Tart Class

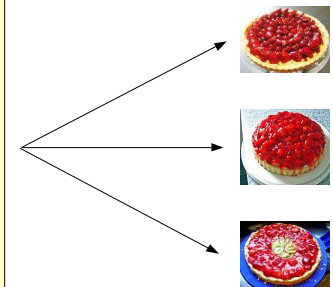
class (Recipe)

Ingredients:
1 qt. fresh strawberries,
3/4 c. sugar
2 tsp. fresh lemon juice
6 baked tart shells
1 1/2 tbsp. cornstarch
1 c. water
1/4 tsp. vanilla

Method:
Wash and hull berries. Mix sugar, cornstarch and salt in a small saucepan. Cook, stirring constantly 6 to 8 minutes or until mixture is clear and thickened. Remove from heat and add lemon juice and vanilla. Cool slightly.

Arrange whole strawberries, stem end down in tart shells. Spoon glaze over the top. Refrigerate to set glaze. Top with whipped cream or sour cream, if desired.

Objects/Instances



Python II

Some Simple Definitions

1. An **object** is simply a reference that happens to know which class it belongs to.
In other words: An object is a variable that belongs to a class.
2. A **class** is simply a package that happens to provide methods to deal with object references.
3. A **method** is a function associated with a class or object. In other words: It's simply a subroutine that expects an object reference (or a package name, for class methods) as the first argument.

Encapsulation

Encapsulation means that the internal representation of an object is generally hidden from view outside of the object's definition.

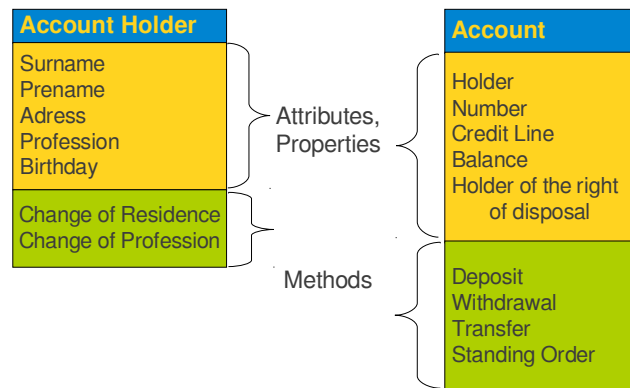
But the methods of the object can directly inspect and manipulate its internal data.

Hiding the internals of the object protects its integrity by preventing users from setting the internal data of the component into an invalid or inconsistent state.

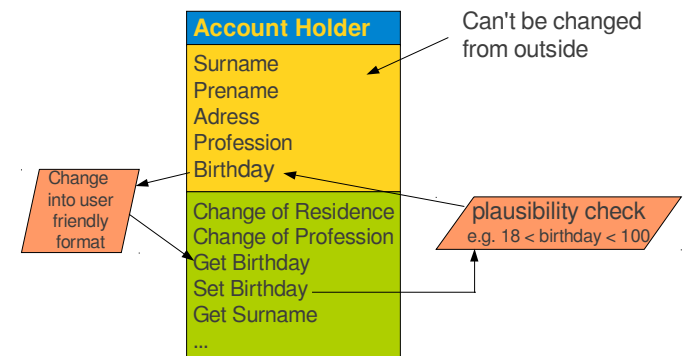
Example:

One has to use the transfer method to make sure that the balance will be all right, i.e. the same amount will be subtracted on one side and added on the other side (account).

Attributes and Methods



Encapsulation of Data



Python II

Instance and Class Methods Instance and Class Attributes

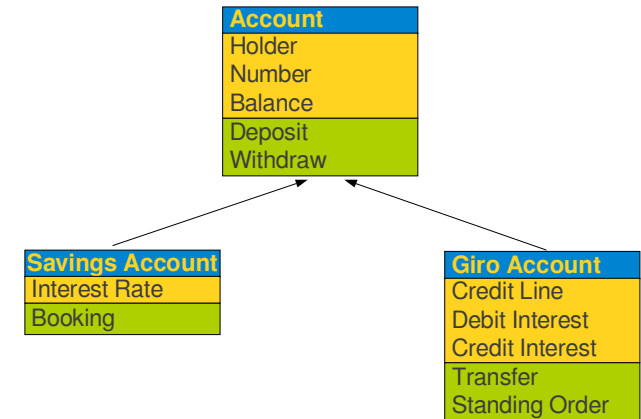
An attribute, which is valid for all instances (objects) of a class, is called a class attribute. Definition for class methods is similar.

Example:

A counter counting the number of instances of a class

Instance attributes are normally different for different instances:
e.g. balance

Inheritance



Different Types of Accounts

It's clear, that our class account is not sufficient for an ordinary bank.

They need different kinds of accounts:

giro account, check account, savings account, deposits etc.

All these accounts have some features in common.

E.g. each account needs an account number, a holder and a balance.

Common methods: deposit and withdrawal

So we have a "common class" or a superclass from which the other classes (giro account, savings account etc.) inherit from.

The Simplest Class in Python

The definition of a new class starts with the keyword **class**:

```
class Account(object):  
    pass
```

This class has no attributes and no methods.

An object or an instance of this (empty) class can be created as follows:

```
>>> Account()  
<__main__.Account object at 0x7f5feca55750>
```

↑ ↑ ↑
Namespace Class Memory Address

Python II

Methods

Methods are just function objects created by `def` statements nested in a class statement's body.

- A method is a function, which is defined inside of a class definition.
- The first parameter of a method is always a reference `self` to the instance, which has called it.

The parameter `self` is only used in the definition of a method. It's not used in the call of the method.

Account Class With Methods

```
class Account(object):  
  
    def transfer(self, target, amount):  
        pass  
  
    def deposit(self, amount):  
        pass  
  
    def withdraw(self, amount):  
        pass  
  
    def balance(self):  
        pass
```

Why „self“

There is a similar concept in Java: But „self“ is called „this“ in Java

Why do we need „self“?

„self“ is a reference to the current instance

But the question remains: Why can't Python administer "self" automatically?

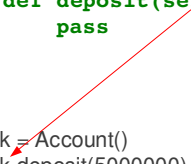
Answer:

It's the way it is, because Guido van Rossum wanted it like this.

Account Class With Methods

```
class Account(object):  
  
    def deposit(self, amount):  
        pass
```

```
>>> k = Account()  
>>> k.deposit(5000000)
```



Python II

Constructor

`__init__` functions like a constructor of other programming languages.

`__init__` is called immediately after an instance of the class is created.

But `__init__` is not the constructor of the class.

But it is close to being a constructor:

- it looks like a constructor
- it's used like one (it's the first code executed in a newly created instance of the class),
- the name sounds like a constructor

But it is incorrect, because the object has already been constructed by the time `__init__` is called, and you already have a valid reference to the new instance of the class.

Destructor

`object.__del__(self)`

This method is called when the instance is about to be destroyed. Therefore it is also called a destructor.

If a base class has a `__del__()` method, the derived class's `__del__()` method, if any, must explicitly call it to ensure proper deletion of the base class part of the instance.

It is not guaranteed that `__del__()` methods are called for objects that still exist when the interpreter exits.

Constructor, Example

```
def __init__(self, holder, number,
balance, credit_line=1500):
    self.Holder = holder
    self.Number = number
    self.Balance = balance
    self.CreditLine = credit_line
```

Example with Both

```
class Greeting(object):
    def __init__(self, name):
        self.name = name
    def __del__(self):
        print "Destructor started"
    def SayHello(self):
        print "Hello", self.name
```

```
>>> execfile("hello_class.py")
>>> x1 = Greeting("Guido")
>>> x2 = x1
>>> del x1
>>> del x2
Destructor started
```

Python II

Instances as Parameters

The method `transfer()` needs an instance `target` of the account class as an argument.

```
def transfer(self, target, amount):
    if(self.Balance - amount < -
self.CreditLine):
        # coverage insufficient
        return False
    else:
        self.Balance -= amount
        target.Balance += amount
        return True
```

Private Members

There is something in our class `Account()`, which we don't like.

It's possible to access the attributes directly from "outside" of the class:

```
>>> k.balance()
9009.7800000000007
>>> k2.Balance
4800.0300000000007
>>> k2.Balance += 25000
>>> k2.Balance
29800.0299999999999
>>>
```

Using the Account Class

```
>>> import Account
>>> k =
Account.Account("Guido", 345267, 10009.78)
>>> k.balance()
10009.7800000000001
>>> k2 = Account.Account("Sven", 345289, 3800.03)
>>> k2.balance()
3800.0300000000002
>>> k.transfer(k2, 1000)
True
>>> k2.balance()
4800.0300000000007
>>> k.balance()
9009.7800000000007
>>>
```

Name Schema for Attributes

Names	Type	Meaning
name	Public	These Attributes are visible from outside and from inside, i.e. they can be read and written like Public, but the developer doesn't want it.
<u>name</u>	Protected	The will not be imported in the namespace by <code>from ... import *</code>
<u><u>name</u></u>	Private	Can't be seen and used from outside

Python II

Properties

With the property-Function it's possible to access private attributes as if they were public:

```
class P(object):
    def __init__(self,x):
        self.__x = x

    def getX(self):
        return self.__x

    def setX(self, x):
        self.__x = x

    x = property(getX,setX)
```

Using the P class:

```
>>> from properties import P
>>> a = P(12)
>>> print a.x
12
>>> a.x = 42
>>> print a.x
42
>>>
```

Dynamical Attributes

Attributes can be dynamically created at runtime:

```
>>> class book(object):
...     pass
...
>>> b1 = book()
>>> b1.title = "Learning Python"
>>> b1.author = "Ben Small"
>>> print b1.title
Learning Python
>>> b2 = book()
>>> print b2.title
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'book' object has no attribute 'title'
>>>
```

Attention:
Potential source
of error!
Attributes might be
misspelled.

Properties without writing access

Without the second argument there is no writing access:

```
class P(object):
    def __init__(self,x):
        self.__x = x

    def getX(self):
        return self.__x

    def setX(self, x):
        self.__x = x

    x = property(getX)
```

Usage:

```
>>> from properties2 import P
>>> a = P(12)
>>> print a.x
12
>>> a.x = 42
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: can't set attribute
>>> a.setX(42)
>>> print a.x
42
>>>
```

Private Members in Account()

The "constructor" of class Account with private Members:

```
def __init__(self, holder, number,
balance,credit_line=1500):
    self.__Holder = holder
    self.__Number = number
    self.__Balance = balance
    self.__CreditLine = credit_line
```

Trying to access the balance directly throws an error:

```
>>> K2.__Balance
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'Account' object has no attribute '__Balance'
```

Python II

bodenseo

Python 2

Static Class Variables

A variable declared inside the class definition, but not inside of a method is called a **class** or a **static variable**.

Instance variables (also called dynamic variables) are defined inside of the methods. They are referred to with "self.<name>"

```
class Account(object):
    counter = 0
    def __init__(self, holder, number,
balance,credit_line=1500):
        Account.counter += 1
        self.__Holder = holder
        self.__Number = number
        self.__Balance = balance
        self.__CreditLine = credit_line
    def __del__(self):
        type(self).counter -= 1
```

bodenseo, Bernd Klein, 09/04/13

Folie: 149

bodenseo

Python 2

Polymorphism

You could try to implement something like this:

```
class NN:
    def abc(self,x):
        pass
    def abc(self,x,y):
        pass
```

This code will run, but not in the intended way.

It's like having two assignments to the same name in sequence:

```
a = 5
a = 7
```

The second definition will be the one overwriting the first one.

bodenseo, Bernd Klein, 09/04/13

Folie: 151

bodenseo

Python 2

Polymorphism

Polymorphism is often defined as the overloading of functions based on the type signatures of their arguments.

This is different in Python as there is no type declaration in Python.

Polymorphism in Python is grounded on object interfaces instead of types.

bodenseo, Bernd Klein, 09/04/13

Folie: 150

bodenseo

Python 2

Varying Numbers of Parameters

A varying number of arguments can be dealt with like this:

```
class NN:
    def abc(self, *args):
        if len(args) == 1:
            pass
        elif len(args) == 2:
            pass
        else:
            pass
```

It's even possible to base the decisions on the type of the argument:

```
if type(arg[0]) == int:
    pass
```

bodenseo, Bernd Klein, 09/04/13

Folie: 152

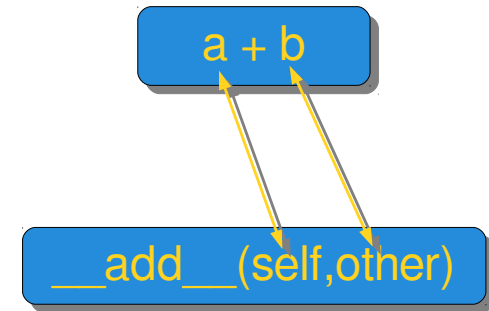
Python II

Polymorphism

The following example shows polymorphism as operator overloading with the „+“-Operator

```
>>> a = "Spam spam spam"
>>> b = a + "!"
>>> 3 + 4
7
```

Operating Principle



Parameter Overloading

Although we can implement all class behaviour as method functions, it is in many cases more convenient to use operator overloading.

- Python provides specially named methods `__x__` to carry out operator overloading, e.g. `__add__` for `+`
- These methods are automatically called if the corresponding operator is found in an expression
- overloaded operators are like built-ins
- most built-in operators can be overwritten

Built-in Methods of Operators

```
+   object.__add__(self, other)
-   object.__sub__(self, other)
*   object.__mul__(self, other)
//  object.__floordiv__(self, other)
/   object.__div__(self, other)
%   object.__mod__(self, other)
**  object.__pow__(self, other[,
modulo])
<< object.__lshift__(self, other)
>> object.__rshift__(self, other)
&   object.__and__(self, other)
^   object.__xor__(self, other)
|   object.__or__(self, other)
```

Python II

Augmented Arithmetic Assignments

```
+= object.__iadd__(self, other)
-= object.__isub__(self, other)
*= object.__imul__(self, other)
/= object.__idiv__(self, other)
//= object.__ifloordiv__(self, other)
%= object.__imod__(self, other)
**= object.__ipow__(self, other[, modulo])
<*= object.__ilshift__(self, other)
>*= object.__irshift__(self, other)
&= object.__iand__(self, other)
^= object.__ixor__(self, other)
|= object.__ior__(self, other)
```

Comparison Methods

```
< object.__lt__(self, other)
<= object.__le__(self, other)
== object.__eq__(self, other)
<>, != object.__ne__(self, other)
> object.__gt__(self, other)
>= object.__ge__(self, other)
```

Unary Operators

```
- object.__neg__(self)
+ object.__pos__(self)
abs() object.__abs__(self)
~ object.__invert__(self)
complex() object.__complex__(self)
int() object.__int__(self)
long() object.__long__(self)
float() object.__float__(self)
oct() object.__oct__(self)
hex() object.__hex__(self)
```

Exercise

Write a class for Fractional Arithmetic, implementing addition, subtraction, multiplication and division. The results of the arithmetic operations should be a fraction which can't be reduced.



Python II

bodenseo

Python 2

Fractional Arithmetic 1

```
class fraction(object):

    def __init__(self,z,n):
        self.z = z
        self.n = n
    def reduce(self):
        a=self.z
        b=self.n
        while b != 0:
            a,b = b,a%b
        self.z = self.z/a
        self.n = self.n/a
    def __add__(self,other):
        s =
        fraction(self.z*other.n+other.z*self.n,
                  self.n*other.n)

        s.reduce()
        return s
```

bodenseo, Bernd Klein, 09/04/13

Folie:161

bodenseo

Python 2

Fractional Arithmetic 3

```
    def __str__(self):
        return str(self.z)+'/'+str(self.n)

    def __cmp__(self,other):
        if self.z*other.n < other.z*self.n: return
        -1
        elif self.z*other.n == other.z*self.n:
        return 0
>>> import fractions
>>> x = fractions.fraction(3,7)
>>> y = fractions.fraction(8,11)
>>> z = x * y
>>> print z
24/77
>>> z = x / y
>>> print z
33/56
>>> x < y
True
>>>
```

bodenseo, Bernd Klein, 09/04/13

Folie:163

bodenseo

Python 2

Fractional Arithmetic 2

```
    def __sub__(self,other):
        d = fraction(1,1)
        d.z = self.z*other.n-other.z*self.n
        d.n = self.n*other.n
        d.reduce()
        return d
    def __mul__(self,other):
        p = fraction(1,1)
        p.z = self.z*other.z
        p.n = self.n*other.n
        p.reduce()
        return p
    def __div__(self,other):
        q = fraction(1,1)
        q.z = self.z*other.n
        q.n = self.n*other.z
        q.reduce()
        return q
```

bodenseo, Bernd Klein, 09/04/13

Folie:162

bodenseo

Python 2

Inheritance

The syntax for a derived class definition looks like this:

```
class DerivedClassName(BaseClassName):
    <statement-1>
    .
    .
    .
    <statement-N>
```

The name BaseClassName must be defined in a scope containing the derived class definition.

bodenseo, Bernd Klein, 09/04/13

Folie:164

Python II

Multiple Inheritance

A class definition with multiple base classes can be defined like this:

```
class DerivedClassName(Base1, Base2, Base3):
```

```
    <statement-1>
```

```
    .  
    .  
    .
```

```
    <statement-N>
```

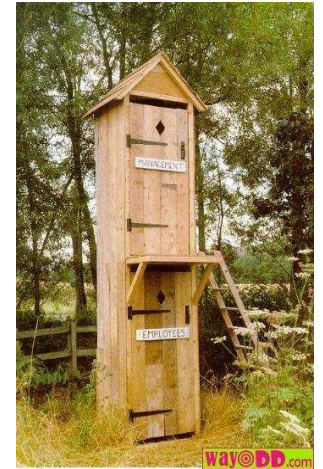
If an attribute is not found in the class DerivedClassName, it is searched for in Base1, then (recursively) in the base classes of Base1, and only if it is not found there, the search continues in Base2, and so

Employee --> Person

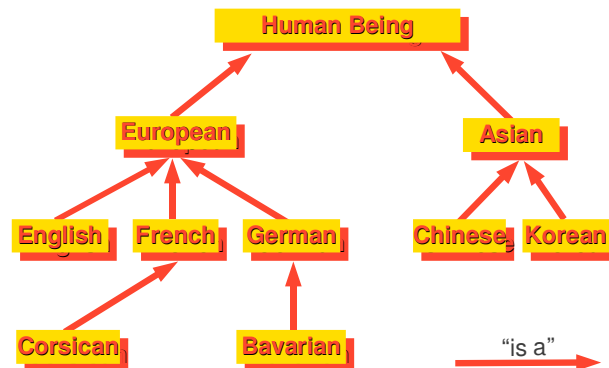
Every Employee is a Person.

We will extend our example with a class "Employee".

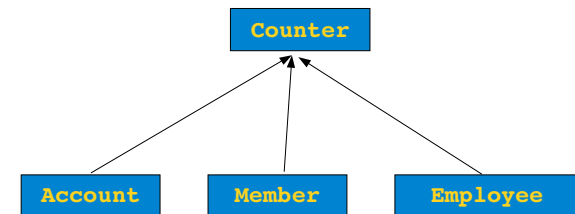
It inherits the methods of Person.



Inheritance



Overview



Python II

Counter Class

```
class Counter(object):
    number = 0

    def __init__(self):
        type(self).number += 1

    def __del__(self):
        type(self).number -= 1

class Account(Counter):
    def __init__(self, account_holder,
                  account_number,
                  balance,
                  account_current=1500):
        Counter.__init__(self)
```

Clock Class, I

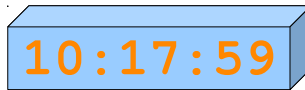
```
class Clock(object):

    def __init__(self, hours=0, minutes=0, sec=0):
        self.__hours = hours
        self.__minutes = minutes
        self.__seconds = sec

    def set(self, hours, minutes, seconds=0):
        self.__hours = hours
        self.__minutes = minutes
        self.__seconds = seconds
```

Exercise

Write a class
clock for a digital clock.



It should implement the following methods:

```
set(hours, minutes, seconds = 9);
tick();
display();
```

The method `tick()` should “tick” for a second, i.e. the
time should be advanced by one second:
11:59:59 → 12:00:00

Clock Class, III

```
def display(self):
    print("%d:%d:%d" % (self.__hours,
                        self.__minutes, self.__seconds))

    def __str__(self):
        return "%2d:%2d:%2d" % (self.__hours,
                                self.__minutes, self.__seconds)

if __name__ == "__main__":
    x = Clock()
    print(x)
    for i in xrange(10000):
        x.tick()
    print(x)
```

Python II

Clock Class, II

```
def tick(self):
    """ Time will be advanced by one second
    """
    if self.__seconds == 59:
        self.__seconds = 0
        if (self.__minutes == 59):
            self.__minutes = 0
            self.__hours = 0 if self.__hours==23
        else self.__hours+=1
    else:
        self.__minutes += 1;
    else:
        self.__seconds += 1;
```

Cooperative Classes

An example for a traditional non-cooperative super call:

```
class B:
    def m(self):
        print "B here"

class C(B):
    def m(self):
        print "C here"
        B.m(self)
```

The method m() of B is extended in class C.

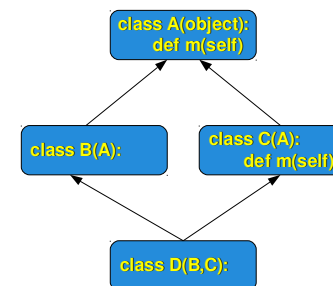
Multiple Inheritance

10:17:59

< January >				< 2011 >			
	Mon	Tue	Wed	Thu	Fri	Sat	Sun
52	27	28	29	30	31	1	2
1	3	4	5	6	7	8	9
2	10	11	12	13	14	15	16
3	17	18	19	20	21	22	23
4	24	25	26	27	28	29	30
5	31	1	2	3	4	5	6



Cooperative Classes



The diagram shows that B and C inherit from A that D inherits from B and C.

Let's assume that a method m() of A is overwritten in C. m() will not be changed in B.

If we apply the method m() on an instance of the class D, m() of C and not m() of A will be used.

That's not what we want: This way all extensions of m() of B will have no effect in D.

Python II

A Simple Example

```
class A(object):
    def m(self):
        print "m in A"
class B(A):
    pass
class C(A):
    def m(self):
        print "m in C"
class D(B,C):
    pass
```

The actions of method m() of A will not be executed in c.m(), i.e.

```
print "m in A"
```

Using the classes:

```
>>> from ABCD import
A,B,C,D
>>> a = A()
>>> a.m()
m in A
>>> b = B()
>>> b.m()
m in A
>>> c = C()
>>> c.m()
m in C
>>> d = D()
>>> d.m()
m in C
```

Another Problem

We extend the previous example with a method m() in B:

Using these classes:

```
class A(object):
    def m(self):
        print "m in A"
class B(A):
    def m(self):
        A.m(self)
        print "m in B"
class C(A):
    def m(self):
        A.m(self)
        print "m in C"
class D(B,C):
    pass
```

```
>>> from ABCD3 import
A,B,C,D
>>> d = D()
>>> d.m()
m in A
m in B
>>>
```

We might expect the D inherits both the methods m() of B and of C, but Python uses only the first class, i.e. B.

Example

The method m() of the previous example overloads the method m() of A. In most cases we want an extension, i.e. first m() of A should be executed and afterwards m() of C.

```
def m(self):
    A.m(self)
    print "m in C"
```

The output:

```
>>> from ABCD import A,B,C,D
>>> c = C()
>>> c.m()
m in A
m in C
```

"Solution" for the problem

We could overload m() in D as well, and call m() of B and C explicitly:

```
class A(object):
    def m(self):
        print "m in A"
class B(A):
    def m(self):
        A.m(self)
        print "m in B"
class C(A):
    def m(self):
        A.m(self)
        print "m in C"
class D(B,C):
    def m(self):
        B.m(self)
        C.m(self)
```



Python II

bodenseo

Python 2

Traditional Approach

```
class A(object):
    def m(self):
        print "do what has to be done for A"
class B(A):
    def _m(self):
        print "do what has to be done for B"
    def m(self):
        self._m()
        A.m(self)
class C(A):
    def _m(self):
        print "do what has to be done for C"
    def m(self):
        self._m()
        A.m(self)
class D(B,C):
    def m(self):
        A.m(self)
        B._m(self)
        C._m(self)
        print "do what has to be done for D"
```

bodenseo, Bernd Klein, 09/04/13

Folie: 181

bodenseo

Python 2

Method Resolution Order

Python has a class attribute `__mro__` for every class.
It's a tuple, containing the class itself and all the superclasses.

```
>>> D.__mro__
(<class 'ABCD_trad.D'>,
 <class 'ABCD_trad.B'>,
 <class 'ABCD_trad.C'>,
 <class 'ABCD_trad.A'>,
 <type 'object'>)
```

bodenseo, Bernd Klein, 09/04/13

Folie: 183

bodenseo

Python 2

Using the Previous Examples

The previous approach works as we can see:

```
>>> from ABCD_trad import A,B,C,D
>>> d = D()
>>> d.m()
do what has to be done for A
do what has to be done for B
do what has to be done for C
do what has to be done for D
```

bodenseo, Bernd Klein, 09/04/13

Folie: 182

bodenseo

Python 2

super()

A call to `super(D, self)` returns a super-Object.

It finds the next class following D in `self.__class__.__mro__`

```
>>> D.__mro__
(<class 'ABCD_super.D'>, <class 'ABCD_super.B'>,
 <class 'ABCD_super.C'>, <class 'ABCD_super.A'>, <type 'object'>)
```

```
class A(object):
    def m(self):
        print "do what has to be done for A"
class B(A):
    def m(self):
        print "do what has to be done for B"
        super(B, self).m()
class C(A):
    def m(self):
        print "do what has to be done for C"
        super(C, self).m()
class D(B,C):
    def m(self):
        print "do what has to be done for D"
        super(D, self).m()
```

→ Class C

→ Class A

→ Class B

bodenseo, Bernd Klein, 09/04/13

Folie: 184

Python II



boderseo

Python 2

**THANK YOU
VERY MUCH
FOR YOUR
ATTENTION!**

boderseo, Bernd Klein, 09/04/13

Folie: 185